Design and Analysis of Algorithms

Algorithmic Classes of Complexity
(P, NP, NP-Completeness, NP-Hardness)

Dealing with Hard Problems

- What to do when we find a problem that looks hard…?

- Sometimes we can prove a strong lower bound… (but not usually)

Types of Problems

Optimization Problem

- Optimization problems are those for which the objective is to maximize or minimize some values. For example:
  - Finding the minimum number of colors needed to color a given graph.
  - Finding a tour with minimal total weight on its edges.

Decision Problem

- There are many problems for which the answer is a Yes or a No.
- For example:
  - Whether a given graph can be colored by only 4-colors?
  - Whether there is a tour having total weight no greater than d?
  - Does a text T contain a pattern P?
  - Does an instance of 0/1 Knapsack have a solution with benefit at least K?
  - Does a graph G have an MST with weight at most K?
  - Does a given graph G have an Hamilton Circuit?

Determinism vs. Nondeterminism

- Nondeterministic algorithms produce an answer by a series of “correct guesses”

- Deterministic algorithms (like those that a computer executes) make decisions based on information.

Polynomial-Time Decision Problems

- To simplify the notion of “hardness,” we will focus on the following:
  - Polynomial-time as the cut-off for efficiency
  - Decision problems: output is 1 or 0 (“yes” or “no”)

What is polynomial-time?

- A ***polynomial-time algorithm*** is one whose worst-case time complexity is bounded above by a **polynomial function** of its input size.

- Polynomial-time: running time is $O(n^k)$, where *k* is a constant.

The Complexity Class P

- A **complexity class** is a collection of problem
- P is the complexity class consisting of all problems that are solved by **polynomial-time** algorithms
- More specifically, they are problems that can be solved in time $O(n^k)$ for some constant k, where n is the size of the input to the problem
- The key is that n is the **size of input**
- For each problem L in P there is a polynomial-time decision algorithm A for L.
  - If n=|x|, for x in L, then A runs in polynom(n) time on input x.
  - The function p(n) is some polynomial

The Complexity Class NP

non deterministic polynomial

- NP = Non-Deterministic polynomial time
- We say that a non-deterministic algorithm A **accepts/solve** a string x if there exists some sequence of choose operations that causes A to output "yes" on input x.

- NP means verifiable in polynomial time

would it give yes or no

- NP is the complexity class consisting of all problems accepted/solved by **polynomial-time non-deterministic** algorithms.

the time isnt deterministic like Las Vegas

P

Problems solved in Polynomial time

NP

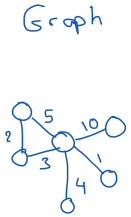
Problems solved in Polynomial time using non-deterministic algorithm

$P \cap NP = \emptyset$ False

$P \subseteq NP$ True

↑ subset

NP example



Can the min spanning tree (MSP) be at least 15?

it could say no or yes depending on the algorithm

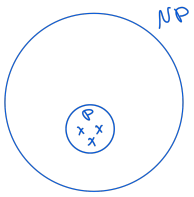
- Problem: Decide if a graph has an MST of weight K

- Algorithm:
  1. Non-deterministically choose a set T of n-1 edges
  2. Test that T forms a spanning tree
  3. Test that T has weight at most K

- Analysis: Testing takes O(n+m) time, so this algorithm runs in polynomial time.

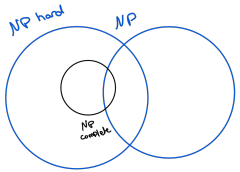
What are P and NP?

- Based on the definition of P and NP, which statements are correct?
  - "NP means non-polynomial"
    - No!
  - $P \cap NP = \emptyset$
    - No. $P \subseteq NP$
  - A P problem is also a NP problem
    - Yes. $P \subseteq NP$



NP-Hard

- A problem (language) L is **NP-hard** if every problem in NP can be reduced to L in polynomial time.
- That is, for each problem M in NP, we can **transform** it in polynomial time to L (and prove the relationship)
- NP-hard problems *do not have to be in NP*, and *they do not have to be decision problems.*
- Cook, Karp, and others, defined the class of NP-hard problems, which most people believe cannot be solved in polynomial time, even though nobody can prove a super-polynomial lower bound.

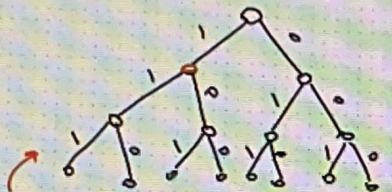


Free

$x_1 : 0$
$x_2 : 1$
$x_3 : 1$

$(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \longrightarrow \square$

2^n
n: # of variables.



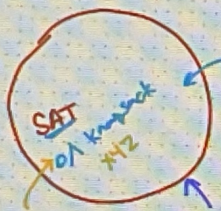
NP-Hard

reduction in polynomial time

reduced

0/1 knapsack

NP-Hard



Show that a problem is related to SAT.

Boolean Satisfiability Problem

0/1 knapsack

0/1 knapsack ∝ SAT

XYZ → 0/1 knapsack

XYZ

any other problem that can be solved using NP-hard

X → y y → z

X → z

NP-complete

- A NP-complete problem(NPC) is
  - a NP problem
  - harder than or equal to all NP problems (NP-hard)
- Why we study NPC?
  - If you see a problem is NPC, you can stop from spending time and energy to develop a fast polynomial-time algorithm to solve it.
  - Reduce it to a known NP-hard problem and write a non-deterministic polynomial-time algorithm, and your work is complete

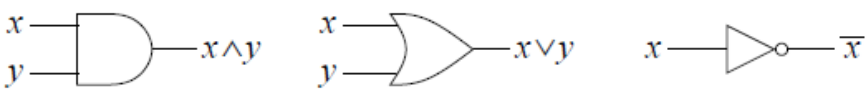
What if a NPC problem needs to be solved?

- Buy a more expensive machine and wait
  - (could be 1000 years)
- Turn to approximation algorithms
  - Algorithms that produce near optimal solutions

Circuit satisfiability

- Circuit satisfiability is a good example of a problem that we don't know how to solve in polynomial time.

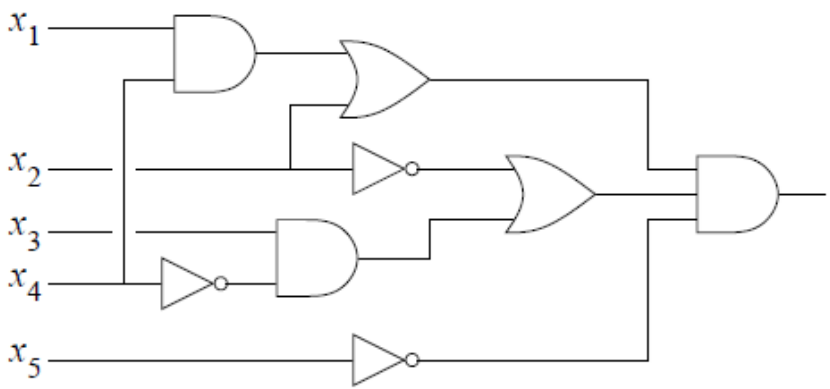
- The input is a boolean circuit:
  - a collection of **and**, **or**, and **not** gates connected by wires.



An AND gate, an OR gate, and a NOT gate.

Circuit satisfiability

- The input to the circuit is a set of m boolean (true/false) values $x_1; \ldots ; x_m$.
- The output is a single boolean value.



A boolean circuit. Inputs enter from the left, and the output leaves to the right.

Circuit satisfiability

- The circuit satisfiability problem asks, given a circuit, whether there is an input that makes the circuit output True, or conversely, whether the circuit always outputs False.
- Nobody knows how to solve this problem faster than just trying all 2^m possible inputs to the circuit, but this requires exponential time.
- On the other hand, nobody has ever proved that this is the best we can do;
- Maybe there's a clever algorithm that nobody has discovered yet!

SAT ((exponential))

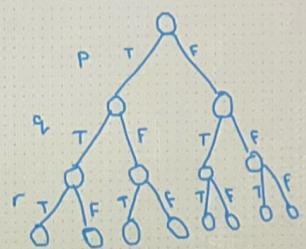
p	q	r	$(p \wedge q)$	$(p \wedge q) \vee r$	$(p \wedge q) \vee r$
T	T	T	T	T	
T	T	F	T	T	
T	F	T	F	T	
T	F	F	F	F	
F	T	T	F	T	
F	T	F	F	F	
F	F	T	F	T	
F	F	F	F	F	

$2^3 = 8$

2^4

2^5

2^n

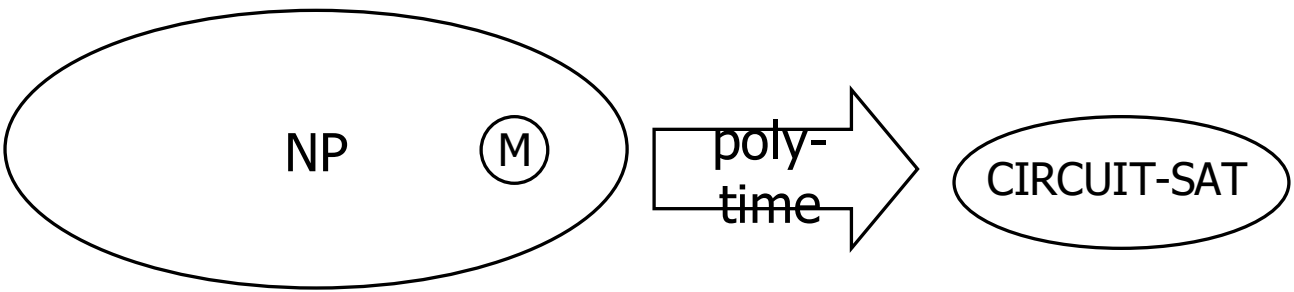


Cook's Theorem

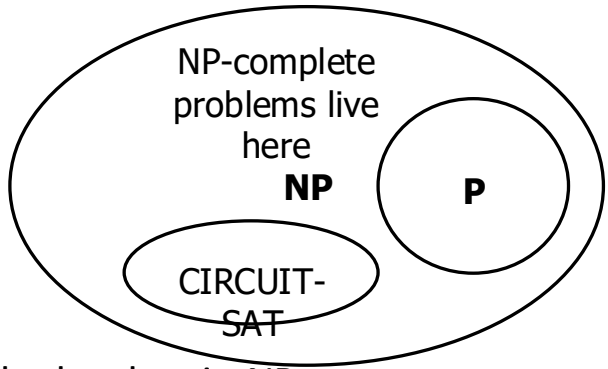
- The following theorem was proved by Steve Cook in 1971:
  - **Circuit satisfiability is NP-hard.**
- To prove that any problem is NP-hard, we use a reduction argument:
  - **To prove that problem A is NP-hard, reduce a known NP-hard problem to A.**

Cook-Levin Theorem

- **CIRCUIT-SAT is NP-complete.**
  - We already showed it is in NP.
- To prove it is NP-hard, we have to show that every language in NP can be reduced to it.
- The Theorem of Cook-Levin proof it !!



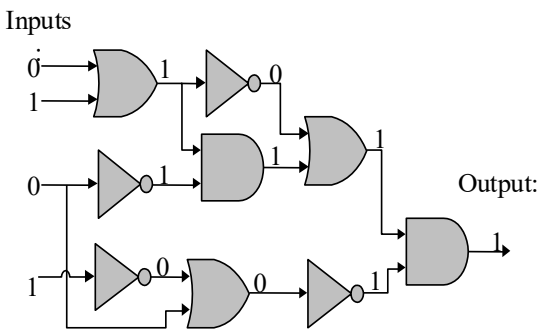
Some Thoughts about P and NP



- Belief: P is a proper subset of NP.
- Implication: the NP-complete problems are the hardest in NP.
- Why: Because if we could solve an NP-complete problem in polynomial time, we could solve every problem in NP in polynomial time.
- That is, if an NP-complete problem is solvable in polynomial time, then **P=NP**.
- Since so many people have attempted without success to find polynomial-time solutions to NP-complete problems, showing your problem is NP-complete is equivalent to showing that a lot of smart people have worked on your problem and found no polynomial-time algorithm.

Problem Reduction

- A language M is polynomial-time **reducible** to a language L if an instance x for M can be transformed in polynomial time to an instance x' for L such that x is in M if and only if x' is in L.
  - Denote this by M→L.
- A problem (language) L is **NP-hard** if every problem in NP is polynomial-time reducible to L.
- A problem (language) is **NP-complete** if it is in NP and it is NP-hard.
- CIRCUIT-SAT is NP-complete:
  - CIRCUIT-SAT is in NP
  - For every M in NP, M → CIRCUIT-SAT.



Transitivity of Reducibility

- If A → B and B → C, then A → C.
  - An input x for A can be converted to x' for B, such that x is in A if and only if x' is in B. Likewise, for B to C.
  - Convert x' into x'' for C such that x' is in B iff x'' is in C.
  - Hence, if x is in A, x' is in B, and x'' is in C.
  - Likewise, if x'' is in C, x' is in B, and x is in A.
  - Thus, A → C, since polynomials are closed under composition.

like $O(n^2)$ for one problem and $O(n^3)$ for another

$O(n^2) + O(n^3) = O(n^3)$ which is still Polynomial

Formula Satisfiability Problem (SAT)

- A **Boolean formula** is a formula where the variables and operations are Boolean (0/1):
- Example:
  - (a+b+¬d+e)(¬a+¬c)(¬b+c+d+e)(a+¬c+¬e)
  - OR: +, AND: (times), NOT: ¬

- SAT: Given a Boolean formula S, is S satisfiable, that is, can we assign 0's and 1's to the variables so that S is 1 ("true")?
  - Easy to see that SAT is in NP:
    - Non-deterministically choose an assignment of 0's and 1's to the variables and then evaluate each clause. If they are all 1 ("true"), then the formula is satisfiable.

- Is it NP-Hard?

SAT is NP-hard and since we know its NP=D SAT is NP complete

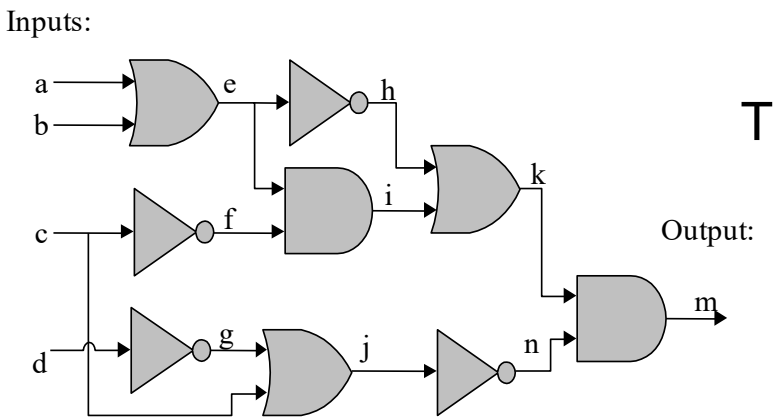
Formula Satisfiability Problem (SAT)

- To show that SAT is NP-hard, we need to give a reduction from a known NP-hard problem.

- The only problem we know is NP-hard so far is **circuit satisfiability**.

Formula Satisfiability Problem (SAT)

- Reduce CIRCUIT-SAT to SAT.
  - Given a Boolean circuit, make a variable for every input and gate.
  - Create a sub-formula for each gate, characterizing its effect. Form the formula as the output variable AND-ed with all these sub-formulas:
    - Example: m((a+b)↔e)(c↔¬f)(d↔¬g)(e↔¬h)(ef↔i)…



The formula is satisfiable if and only if the Boolean circuit is satisfiable.

3SAT

- The SAT problem is still NP-complete even if the formula is a conjunction of disjuncts, that is, it is in conjunctive normal form (CNF):
  - A boolean formula is in **Conjunctive Normal Form (CNF)** if it is a conjunction (and) of several clauses, each of which is the disjunction (or) of several literals, each of which is either a variable or its negation.

$$\overbrace{(a \vee b \vee c \vee d)}^{\text{clause}} \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b})$$

- The SAT problem is still NP-complete even if it is in CNF and every clause has just 3 literals (a variable or its negation):
  - (a+b+¬d)(¬a+¬c+e)(¬b+d+e)(a+¬c+¬e)
- Reduction from SAT (See §13.3.1).
- In 1971, Stephen Cook published a paper proving that if CNF-Satisfiability is in P, then P = NP.

Cook's Theorem

- In 1971 Stephen Cook managed to find a problem that is *NP*-complete.
- The following theorem states his result:
- **CNF-Satisfiability is NP-complete**
- *The proof can be found in Cook (1971) and in Garey and Johnson (1979).*

Cook's Theorem

- The proof does not consist of reducing every problem in *NP* individually to CNF-Satisfiability!
  - If this were the case, whenever a new problem in *NP* was discovered, it would be necessary to add its reduction to the proof.
- The proof **exploits common properties of problems in *NP*** to show that any problem in this set must reduce to CNF-Satisfiability.

Vertex Cover

- A vertex cover of a graph is a set of vertices that touches every edge in the graph.

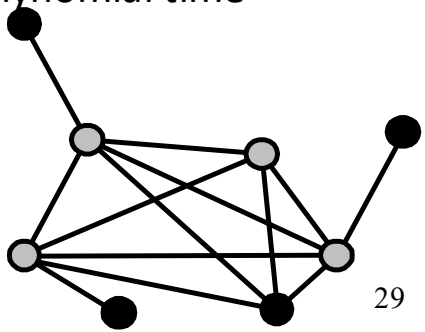
- The Vertex-Cover problem is to find the smallest vertex cover in a given graph.

Vertex Cover

- A vertex cover of graph G=(V,E) is a subset W of V, such that, for every edge (a,b) in E, a is in W or b is in W.
- VERTEX-COVER: Given an graph G and an integer K, is does G have a vertex cover of size at most K?
- Vertex-cover is NP-Complete, to prove that, you need:

1. Show that it is NP
   a. Vertex-cover is in NP: Non-deterministically choose a subset W of size K and check that every edge is covered by W.
   b. It is NP because we can verify the given solution in poly-time

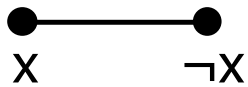
1. Show that it is NP-hard
   a. Reduce a well-known NP-hard problem (CNF Sat) to the Vertex-cover problem, this reduction should be done in polynomial time



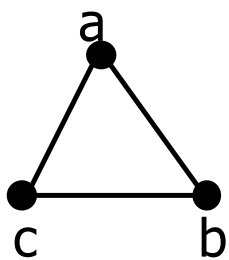
Vertex-Cover is NP-complete

Reduce 3SAT to VERTEX-COVER:

1. Come up with S which is a Boolean formula in CNF with each clause having 3 literals, and 3 clauses.
2. Convert the boolean expression to a graph:
   a. For each variable x, create a node for x and ¬x, and connect these two *pairs*



   a. For each clause (a+b+c), create a *triangle* and connect these three nodes.

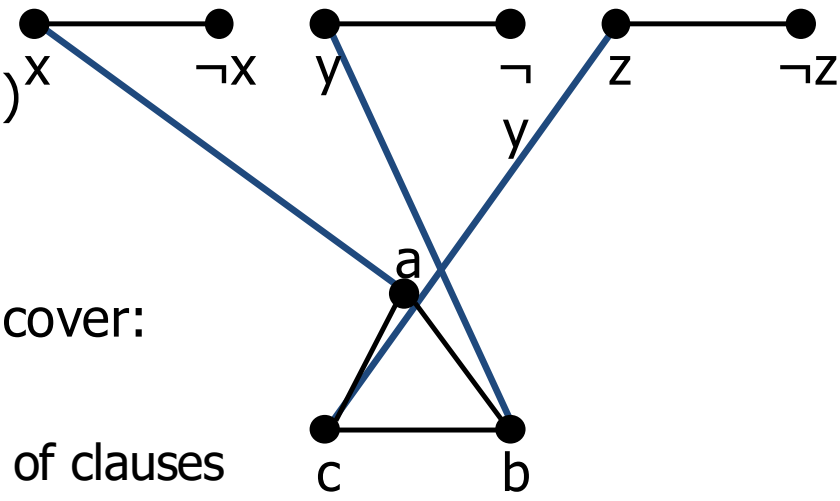


Vertex-Cover is NP-complete

Completing the construction (step 2)

c. Connect each literal in a clause triangle to its copy in a variable pair.

- E.g., a clause (x+y+z)



3. Check if you get k vertex-cover:

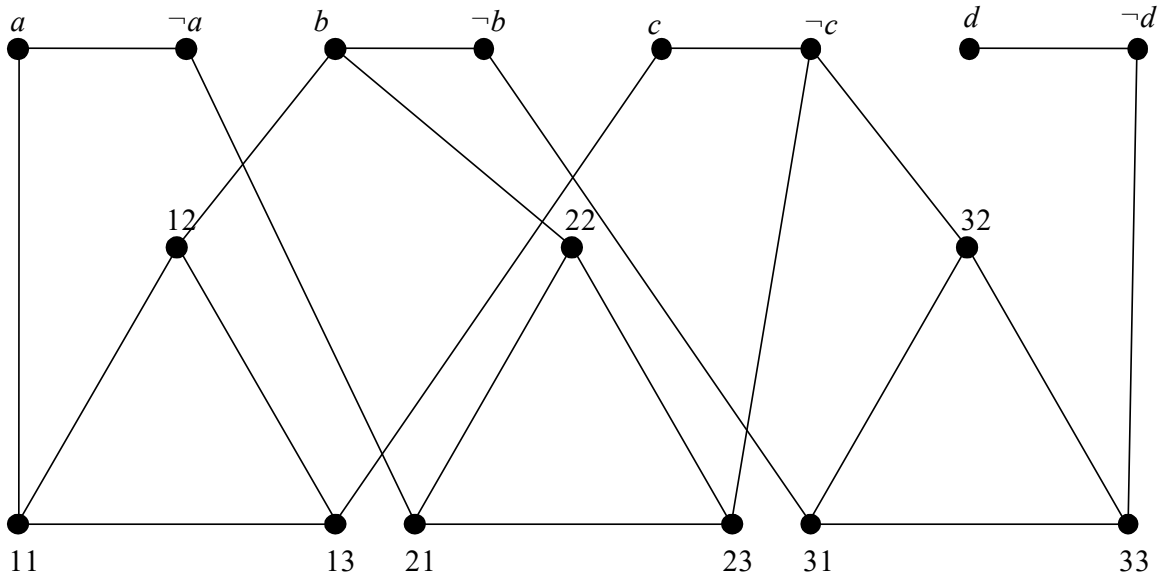
- Set K=n+2m

  n=# of variables, m=# of clauses

4. Mark the selected literals in the selected Vertex-cover as True. Fill these values in the Boolean formula. If the formula = True, Then, the formula is CNF-Sat. Since CNF-Sat is NP-hard, the Clique is NP-hard too.

Vertex-Cover is NP-complete

- Example: (a+b+c)(¬a+b+¬c)(¬b+¬c+¬d)
- Graph has vertex cover of size K=4+6=10 iff formula is satisfiable.



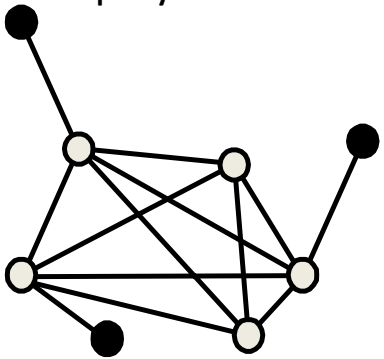
Clique

- A **clique** of a graph G=(V,E) is a subgraph C that is fully-connected (every pair in C has an edge).
- CLIQUE: Given a graph G and an integer K, is there a clique in G of size at least K?
- CLIQUE is NP-Complete, to prove that, you need:

1. Show that it is NP
    a. NP: Write a polynomial-time verification algorithm for this problem

1. Show that it is NP-hard
    a. Reduce a well-known NP-hard problem (CNF Sat) to the this problem, this reduction should be done in polynomial time

This graph has a clique of size 5



Clique as NP-Hard

Reduce CNF-Sat to Clique:

1. Come up with S which is a Boolean formula in CNF with each clause having k clauses.
2. Convert the boolean expression to a graph as follow:
   a. V = {(y, i): y is a literal in clause Ci}
   b. E = {((y,i) , (z, j)): i != j and ¬z != y}
3. Check if you get Clique of size k as maximum
4. Mark the selected literal in the selected clique as True. Fill these values in the Boolean formula. If the formula = True, Then, the formula is CNF-Sat. Since CNF-Sat is NP-hard, the Clique is NP-hard too.

Some Other NP-Complete Problems

- SET-COVER: Given a collection of m sets, are there K of these sets whose union is the same as the whole collection of m sets?
  - NP-complete by reduction from VERTEX-COVER
- SUBSET-SUM: Given a set of integers and a distinguished integer K, is there a subset of the integers that sums to K?
  - NP-complete by reduction from VERTEX-COVER

Some Other NP-Complete Problems

- 0/1 Knapsack: Given a collection of items with weights and benefits, is there a subset of weight at most W and benefit at least K?
  - NP-complete by reduction from SUBSET-SUM
- Hamiltonian-Cycle: Given an graph G, is there a cycle in G that visits each vertex exactly once?
  - NP-complete by reduction from VERTEX-COVER
- Traveling Saleperson Tour: Given a complete weighted graph G, is there a cycle that visits each vertex and has total cost at most K?
  - NP-complete by reduction from Hamiltonian-Cycle.

To Prove that S in NP-Complete:

1. Show ***S*** is in **NP**
2. Select a known **NP**-complete problem ***R***
   - *Since* **R** *is* ***NP****-complete, all problems in* ***NP*** *are reducible to* **R**
3. Show how ***R*** can be poly. reducible to ***S***
   - *Then all problems in* ***NP*** *can be poly. reducible to* **S** *(because polynomial reduction is transitive)*
4. Therefore ***S*** is **NP**-complete

Become Famous! ☺

To get famous in a hurry, for any NP-Complete problem:

- ***Design a Polynomial time Algorithm***

They'll be naming buildings after you before you are dead!